

Chapter 8. Deadlocks

학습 목표 및 학습 내용

▶ 학습 목표

- ▶ 데드락의 개념을 설명할 수 있다.
- ▶ 데드락의 발생 조건을 설명할 수 있다.
- ▶ 데드락의 처리 방안을 설명할 수 있다.

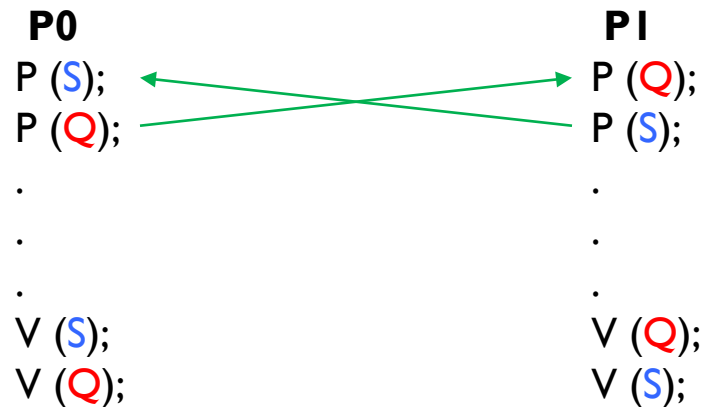
▶ 학습 내용

- ▶ Deadlock concept
- ▶ Deadlock characterization
- ▶ Methods for handling deadlocks
- ▶ Deadlock prevention
- ▶ Deadlock avoidance
- ▶ Deadlock detection and recovery

What is deadlock?

► A semaphore usage example

Let **S** and **Q** be two semaphores initialized to 1.



► **Deadlock**

- For a set of blocked processes, each is **holding a resource** and **waiting to acquire a resource held** by another process in the set.

Deadlock Characterization

▶ System modeling

- ▶ Processes: $P_1, P_2, \dots, P_n$.
- ▶ Resource types: $R_1, R_2, \dots, R_m$
 - ▶ CPU cycles, memory space, I/O devices, files, etc.
- ▶ Each resource type R_i has W_i instances.
- ▶ Each process utilizes a resource as follows.
 - ▶ request
 - ▶ use
 - ▶ release

Deadlock Characterization

- ▶ **Deadlock can arise if four conditions hold simultaneously.**
 - ▶ **Mutual exclusion**
 - ▶ Only one process at a time can use a resource.
 - ▶ **Hold and wait**
 - ▶ A process holding at least one resource is waiting to acquire additional resources held by other processes.
 - ▶ **No preemption**
 - ▶ A resource can be released only voluntarily by the process holding it.
 - ▶ **Circular wait**
 - ▶ There exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that
 - P_0 is waiting for a resource that is held by P_1 ,
 - P_1 is waiting for a resource that is held by P_2 ,
 - ...,
 - P_{n-1} is waiting for a resource that is held by P_n , and
 - P_n is waiting for a resource that is held by P_0 .

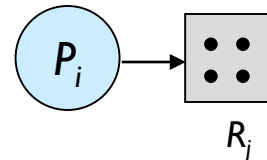
Deadlock Characterization

▶ Resource allocation graph

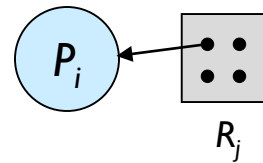
- ▶ V is partitioned into two types.
 - ▶ $P = \{P_1, P_2, \dots, P_n\}$, set of **all processes** in the system.
 - ▶ $R = \{R_1, R_2, \dots, R_m\}$, set of **all resource types** in the system.
- ▶ E is also partitioned into two types.
 - ▶ $P_i \rightarrow R_j$: request
 - ▶ $R_j \rightarrow P_i$: assignment

▶ Examples

- ▶ P_i requests an instance of R_j

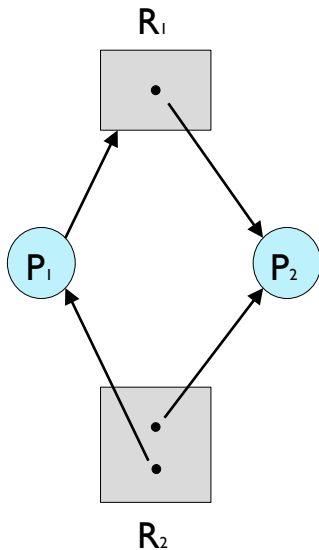


- ▶ P_i is holding an instance of R_j

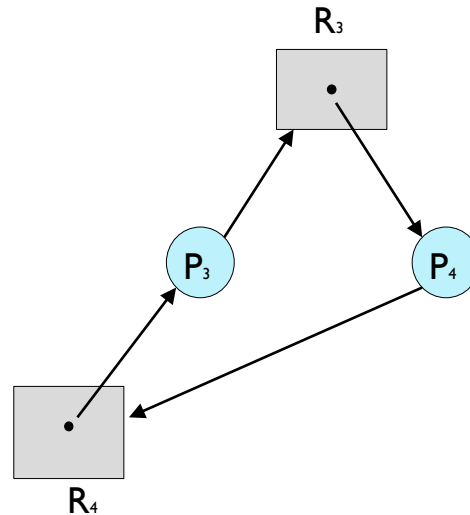


Deadlock Characterization

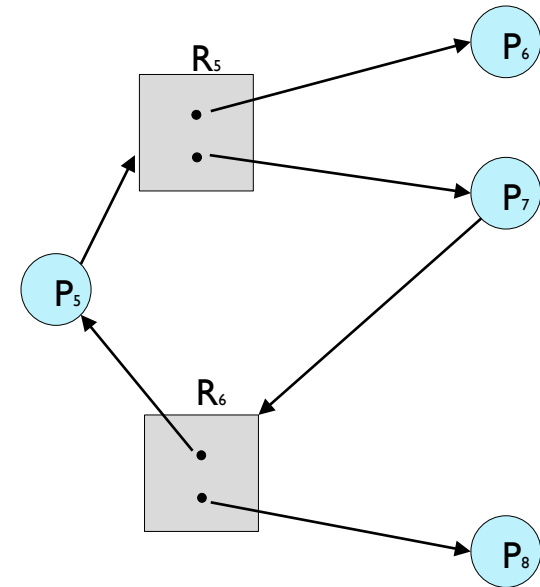
► Deadlock examples with resource allocation graph



(a) deadlock(X)



(b) deadlock (O)



(c) deadlock (X)

If graph contains **no cycles**, **no deadlock**.

If graph contains **a cycle**,

if **only one instance** per resource type, **deadlock**.

if **several instances** per resource type, **possibility of deadlock**.

Methods for Handling Deadlocks

▶ **Prevention**

- ▶ The system will **never enter a deadlock state**.
- ▶ It ensures that **at least one of four conditions cannot hold**.

▶ **Avoidance**

- ▶ The system will **never enter a deadlock state**.
- ▶ It requires **a priori information** about how each process will utilize resources.

▶ **Detection and recovery**

- ▶ It allows the system to enter a deadlock state and then recover.

Deadlock Prevention

- ▶ Ensures that **at least one of four conditions cannot hold.**
- ▶ **Mutual Exclusion**
 - ▶ is **impossible** to deny the mutual exclusion.
- ▶ **Hold and Wait**
 - ▶ requires process to request and be allocated **all its resources** before it begins **execution**.
- ▶ **No Preemption**
 - ▶ If a process holding some resources requests another resource that **cannot be immediately allocated** to it, then all resources currently being held are **released**.
 - ▶ Process will be **restarted only when** it can regain **its old resources** as well as the **new ones** that it is requesting.
- ▶ **Circular Wait**
 - ▶ imposes **a total ordering of all resource types**, and requires that each process requests resources **in an increasing order of enumeration**.

P0	P1
P (S);	P (S);
P (Q);	P (Q);

Deadlock Avoidance

▶ Deadlock avoidance algorithm

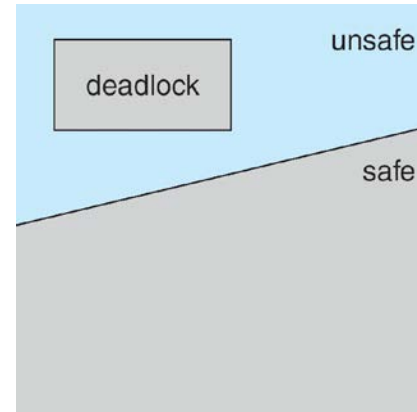
- ▶ When a process requests an available resource, system must decide **if the immediate allocation leaves the system in a safe state**.
 - ▶ A state is **in safe**, if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- ▶ It requires that the system has some additional **a priori information available**.

▶ If a system is **in safe state**,

- ▶ **No deadlocks.**

▶ If a system is **in unsafe state**,

- ▶ **Possibility of deadlock.**



Deadlock Avoidance

▶ Resource allocation graph

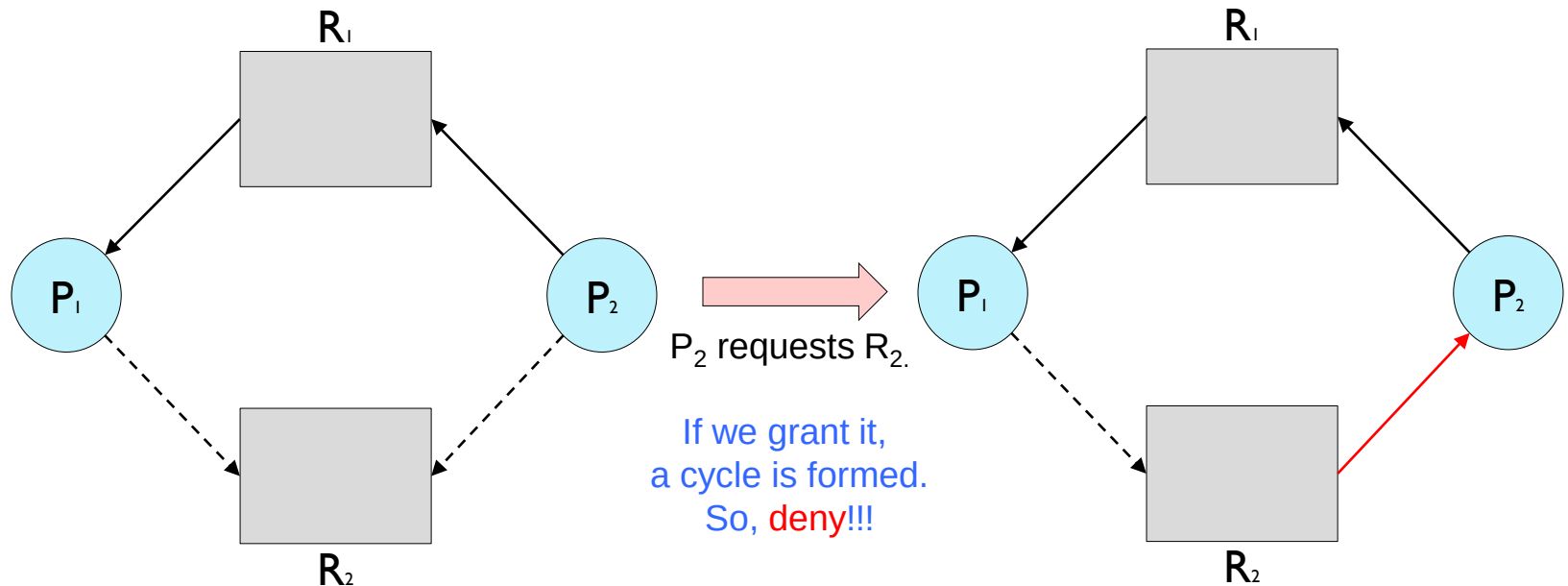
- ▶ **Claim edge** $P_i \rightarrow R_j$
 - ▶ P_j **may request** R_j .
 - ▶ It is represented by a dashed line.
- ▶ **Claim edge** is converted to **request edge**,
 - ▶ when a process **requests** a resource.
- ▶ **Request edge** is converted to **assignment edge**,
 - ▶ when the resource **is allocated** to the process.
- ▶ **Assignment edge** is reconverted to **claim edge**,
 - ▶ when a resource **is released** by a process.

▶ [Note]

- ▶ When multiple instances of a resource type, use the banker's algorithm.

Deadlock Avoidance

► Unsafe state example



No cycle → safe → grant
Cycle → unsafe → deny

Deadlock Detection & Recovery

- ▶ **Allow system to enter deadlock state.**
- ▶ **Detection algorithm.**
 - ▶ Periodically invoke an algorithm that searches for a cycle in the graph.
 - ▶ If there is a cycle, there exists a deadlock.
 - ▶ [Note] When multiple instances of a resource type, use more complicated algorithm.
- ▶ **Recovery scheme.**
 - ▶ Process termination
 - ▶ Abort all deadlocked processes.
 - ▶ Abort one process at a time until the deadlock cycle is eliminated.
 - ▶ Resource preemption
 - ▶ Preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

Summary

- ▶ **Deadlock is a situation that processes are waiting indefinitely for event that is caused by waiting process.**
- ▶ **Deadlock can arise if four conditions; mutual exclusion, hold and wait, no preemption, and circular wait hold simultaneously.**
- ▶ **Deadlock prevention ensures that at least one of four conditions cannot hold.**
- ▶ **Deadlock avoidance ensures that a system will never enter an unsafe state.**
- ▶ **Deadlock detection and recovery allows system to enter deadlock state.**